

RAG_HandProj 源码导读

本文是**作品仓库的进阶导读**：带你读懂 RAG_HandProj 的工程代码——每一环为什么这么设计、模块怎么拆、和原理版差在哪。先学原理请看 [RAG-Minimal \(Gitee\)](#) • [GitHub 镜像](#) (156 行单文件教材，零基础向)；读完原理再读本文，代码逐行都能对上。

0. RAG 是什么？一句话

检索增强生成 (Retrieval-Augmented Generation)：回答前，先从你的私有资料里"查资料"，把查到的原文塞进提问，再让模型基于资料回答。

类比：考试允许开卷。裸问答是闭卷（模型只会背书上的，还可能记错）；RAG 是开卷（带着你的笔记进场，翻到相关段落再答题）。

对应源码：`rag/generate.py` 的 `build_answer_prompt()` ——把资料块 + 问题拼成一个"开卷答题指令"。

1. 为什么回答前要先"查资料"

模型的"记忆"是训练时写死的，有两个硬伤：

- **不知道你的事**：你家的知识库、公司的文档、个人笔记，模型从没见过
- **会一本正经地胡说**：记忆模糊时，模型倾向编一个听起来合理的答案（幻觉）

RAG 的思路：不改造模型（那叫微调，贵且慢），而是在**提问时给模型喂它需要的那几页资料**。模型不擅长记忆，但擅长照着资料说话——各干各擅长的事。

这也是本项目"答案可追溯"的底气：每个回答都挂在检索到的原文块上。

2. 第一步：把文档切成块 (chunking)

模型每次能"看"的文字有限（上下文窗口），不能把整本书塞进去。所以先把文档切成小块，提问时只挑相关的几块。

两种切法，各有用处

窗口切（`rag/chunking.py` 的 `_split_window`）：固定 500 字一块，步长 450——下一块开头 50 字和上一块结尾重复。为什么要重叠？防止一句话被从中间切断：一句话前半截在上一块、后半截在下一块，检索就找不到完整语义。重叠 = 给语义边界"垫保险"。

标题结构切（`chunk_markdown`）：按文档标题分节，一个标题一（或几）块。标题是天然的语义边界，切出来的块"话没说完"的风险低很多，而且标题本身是检索时的强命中信号——命中标题基本等于命中内容。

项目实际用的是**两级策略**：标题分节为主，某个节超过 500 字就在节内再窗口切；文档完全没有标题时退化为纯窗口切（兜底分支，保证任何文档都能处理）。

一个反直觉的细节：**文档主标题不能当分节点**。 `# 整本笔记的名字` 只有一个，如果拿它分节，全篇会并成一节被窗口切，标题锚点全丢（这个坑是真实冒烟抓出来的，有对应测试）。

3. 第二步：把文字变成向量 (embedding)

电脑不懂"意思"，但懂数字。嵌入模型把一段文字映射成一串数字（向量），规则是：**意思相近的文字，向量方向也相近**。

- "猫"和"猫咪"的向量挨得很近
- "猫"和"汽车"的向量离得很远

向量是一串数字，数字本身没意义，**方向**才表达语义。

对应源码：`rag/embed.py`——调 llama.cpp 的 `/v1/embeddings` 接口，拿到 1024 维（4B 模型）或 4096 维（8B 模型）的向量。**向量维度和池化方式因模型而异**：换模型前，先到该模型的开源仓库 README 查这两个参数（一般写作 "output dimension / pooling strategy"），以官方说明为准，别拿别的模型的数值套用。

嵌入模型的坑：必须带 `--embedding` 参数（编码模式），还必须带 `--pooling last`（取最后一层隐藏状态做整句向量）。不带池化参数，向量化会出问题。

4. 第三步：存起来 (SQLite)

向量、原文、来源要一起存，提问时才能查。项目用 SQLite（单文件数据库，零配置，适合学习）：

- `chunks` 表：id / 原文 / 向量（BLOB 二进制） / 来源

- `history` 表：问答历史

两个设计点：

- **向量存二进制**（`to_blob`）：比存文本省空间、查询快，取出时还原成列表
- **去重幂等**（`chunk_hash` 唯一键 + `ON CONFLICT DO UPDATE`）：同一文档重复上传不会攒出重复块——重复构建知识库是开发常态，这个保证"跑一万次结果一致"

对应源码：`rag/store.py`。

5. 第四步：余弦相似度检索（核心）

提问来了：先把问题也变成向量，然后和库里每一块的向量比"方向像不像"，取最像的 **top-k 块**（top = 排名最靠前，k = 取几条；top-k 就是"相似度最高的前 k 条"，本项目 k=3，即每次检索取最像的 3 块喂给模型）。

数学上就是余弦相似度：

$$\text{cosine} = (\mathbf{v1} \cdot \mathbf{v2}) / (|\mathbf{v1}| \times |\mathbf{v2}|)$$

点积衡量方向一致性，除以模长抵消长度影响。范围 -1 到 1，越大越像。

为什么用余弦不用"距离"？向量长度受文本长度影响（长文本向量模长大），余弦只看方向，更公平。

对应源码：`rag/store.py` 的 `search()` ——手写循环遍历 + 排序，没用任何向量数据库（理解原理后再上 pgvector 等是升级路线）。

6. 第五步：拼 Prompt，让模型"开卷答题"

把检索到的 top-k 块拼进提问，并明确指令：**根据以下资料回答，不能编造。**

资料1: ...

资料2: ...

问题: ...

请仅依据以上资料回答，不要编造资料外的内容。

模型遵守"不编造"指令时，资料里没有的内容它会诚实说不知道——这其实是好事：**回答可追溯，错了能查到是哪块资料的问题。**

对应源码：`rag/generate.py`。

7. 为什么 RAG 是"性价比最高"的落地路线

路线	成本	见效
微调 (Fine-tuning)	高：要数据、要算力、要调参，一次不一定成	慢，像重新训练一次
蒸馏 (Distillation)	高：需要大模型老师 + 大量标注	慢，接近研发投入
RAG (检索增强)	低：代码 + 本地模型即可	快：上传文档就能用，效果立竿见影

RAG 不碰模型权重，只在外围搭"查资料"的管道——知识变了，换文档即可，模型不用动。这正是本项目选择的路。

8. 下一步升级路线

本项目是"最小可用"的骨架，升级方向（都是标准路径）：

- **向量数据库**：SQLite 手写循环 → pgvector / Chroma / FAISS（海量数据 + 近似检索）
- **重排序 (rerank)**：检索 top-20 → 精排模型挑 top-3（提升命中精度）
- **混合检索**：向量 + 关键词 (BM25) 互补
- **多路召回 + 元数据过滤**：按文档/时间过滤再检索
- **流式输出 + 引用高亮**：回答边生成边标注依据（演示页已埋了"参考源"交互的种子）

附录：跟着 git log 学本项目

`main` 分支的每个合并点 = 一个可验收的知识点：

M1 工具链：uv 环境 + ruff/mypy/pytest + FastAPI 骨架

M2 核心链路：归一化 → 切块 → 向量化 → 存储 → 余弦检索

M3 /ask 全链路：检索 + Prompt 拼装 + 生成，真实模型冒烟

M4 知识库管理：/library 增删查 + 问答历史落库

M5 门面：演示页（答案可追溯）+ 使用说明 + 学习手册

对照提交信息 + 本文每一节，就是一套完整的 RAG 入门课。

